LLM_Introduction_Guide

Introduction to LLMs (Large Language Models)

Large Language Models (LLMs) are neural network systems trained on massive text corpora to understand, generate, translate, and reason about natural language. They power chatbots, search assistants, code generators, summarizers, and more.

1þ ã What an LLM *Is*

Term	Meaning
LLM	A neural language model with **hundreds of millions to trillions of parameters**.
Parameter	A weight inside the network that is learned during training.
Pre training	Unsupervised or self supervised learning on raw text to capture linguistic knowledge.
Fine tuning	Supervised or reinforcement learning steps that adapt a pre trained LLM to a specific task.
RLHF	Reinforcement Learning from Human Feedback, used to align models with human preferences.
Context window	The number of tokens the model can consider at once (e.g., 4 /k, 8 /k, 32 /k tokens).
Prompt	The input text (plus optional instructions) you give the model to elicit a response.

> **Why “large”?** Scaling parameters, dataset size, and compute together yields emergent abilities —reasoning, in context learning, code generation—that smaller models lack.

2þ ã Architecture Basics

2.1 The Transformer (Vaswani et /al., 2017)

Component	Role
Embedding layer	Maps each token !’ dense vector.
Positional encoding	Adds order information (sinusoidal or learned).
Multi head self attention	Lets every token attend to every other token, learning relationships.
Feed forward network (FFN)	Applies non linear transformation per token.
Layer norm & residuals	Stabilise training & allow deep stacking.
Stacked blocks	Typical LLMs have 12–96+ transformer blocks.

Key formula (scaled dot product attention)
\[\text{Attention}(Q,K,V) = \text{softmax}\!\left(\frac{QK^\top}{\sqrt{d_k}}\right)V\]

2.2 Tokenisation

Method	Description
Byte Pair Encoding (BPE)	Merges frequent byte pairs !’ sub word tokens.
WordPiece	Similar to BPE, used by BERT.
SentencePiece / Unigram	Language agnostic; works on raw Unicode.
Byte Level BPE	Operates on raw bytes !’ no OOV tokens (e.g., GPT 2, LLaMA).

2.3 Scaling Laws (Kaplan et /al., 2020)

Empirical studies show loss "H /A·N { E /+ /B·D {] /+ /C·C { ^ where *N* = model size, *D* = dataset size, *C* = compute. Roughly:
- Doubling parameters !’ modest loss reduction.
- Increasing data !’ more consistent quality gains.
- Compute optimal training balances *N* and *D*.

3þ ã Training Pipeline

3.1 Data Collection & Curation
Stage	Typical Sources
Raw crawl	Common Crawl, Wikipedia, books, code repositories (GitHub, StackOverflow).
Filtering	Remove profanity, personal data, duplicate docs, low quality boilerplate.
Deduplication	Use MinHash or locality sensitive hashing to avoid repeats.
Tokenisation	Apply chosen tokenizer; store as integer IDs.

3.2 Pre training Objective
- **Causal Language Modeling (CLM)** – predict next token (used by GPT style models).
- **Masked Language Modeling (MLM)** – predict masked tokens (BERT style).
- **Prefix LM / Seq2Seq** – predict continuation given a prefix (e.g., T5).

3.3 Fine tuning & Instruction Tuning
Approach	Goal
Supervised fine tuning	Train on task specific labeled data (e.g., QA pairs).
Instruction tuning	Feed prompts plus desired completions to make the model follow natural language instructions.
RLHF	Optimize a reward model (created from human preferences) using PPO to improve helpfulness / safety.

3.4 Compute Infrastructure
- **GPUs** (A100, H100, A800) or **TPUs** (v4) for dense training.
- **ZeRO / FSDP** (DeepSpeed, Megatron LM) for model parallelism.
- **Mixed precision (FP16/BF16)** to cut memory & speed up training.

4þ ã Inference & Deployment

4þ ã Prompt Engineering
Technique	Example
Zero shot	`"Translate English to French: I love you."`
Few shot	`"Q: What is the capital of Japan?\nA: Tokyo\n\nQ: Who wrote '1984'?\nA:"`
Chain of thought	`"Solve step  by  step: 12 × 13 = ?"`
System Prompt (OpenAI)	`"You are a helpful, concise assistant."`

4þ ã Sampling Strategies
Method	When to use
Greedy	Deterministic, fast, but may get stuck.
Top k (k=40)	Truncate distribution to *k* most likely tokens.
Top p (nucleus) (p=0.9)	Keep tokens until cumulative prob "e /p.
Temperature (<B , ³ ã Â ó"ã Ò’ Â 6öçG&öÇ2 andomness; <B óÂ ó Ò Ö÷&R FWFW ministic.
Beam Search	Good for translation or summarisation where completeness matters.

4þ ã Optimisation for Production
Technique	Benefit
Quantisation (int8, GPTQ, AWQ)	2 4× speedup, lower memory.
Distillation (TinyLlama, MiniLM)	Smaller student model with comparable performance.
LoRA (Low Rank Adaptation)	Efficient fine tuning by updating low rank matrices only.
Retrieval Augmented Generation (RAG)	Combine LLM with external knowledge bases for up to date answers.
Parallel inference (TensorRT LLM, vLLM)	Multi GPU serving for high throughput.

5þ ã Popular Model Families (2024 2026 snapshot)

Model	Parameters	Release	Notable Traits
GPT 4 (OpenAI)	~1 /T (estimated)	Mar /2023 !' updates 2024	Multimodal (text+image), strong reasoning.
LLaMA /2 (Meta)	7 /B / 13 /B / 70 /B	Jul /2023	Open weights, permissive license.
Mistral 7B (Mistral AI)	7 /B	Sep /2023	High quality + LoRA friendly.
Gemini /1.5 (Google)	up to 1 /T	2024	Efficient scaling, built in safety.
Claude /3 (Anthropic)	100 /B+	Mar /2024	Constitutional AI, focus on alignment.
Falcon 180B (TII)	180 /B	May /2023	Open source, high compute efficiency.
Mixtral 8×7B (Mistral)	8 expert mixture of 7 /B each	Aug /2024	MoE architecture, better scaling at lower cost.
Bloom 176B (BigScience)	176 /B	2022 (still widely used)	Multilingual dataset (59 languages).

> **Tip:** For most hobbyist or research projects, a 7 13 /B model (LLaMA /2 13B, Mistral 7B, Mixtral 8×7B) offers a good balance of capability and compute cost.

6þ ã Hands On: Running an LLM with Ø>Ý /Transformers
Below is a minimal notebook style script (Python /3.10+) that:
1. **Loads** a quantised 7 /B model (Mistral 7B Instruct) via `bitsandbytes`.
2. **Generates** a response with a system prompt and temperature control.
3. **Shows** how to add LoRA adapters for lightweight fine tuning.

```python
# ------------------------------------------------------------
# 1þ ã Install required packages (run once)
# ------------------------------------------------------------
# !pip install -q transformers accelerate bitsandbytes peft

# ------------------------------------------------------------
# 2þ ã Import libraries
# ------------------------------------------------------------
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, GenerationConfig
from peft import PeftModel, PeftConfig   # for LoRA adapters (optional)

# ------------------------------------------------------------
# 3þ ã Load the tokenizer & quantised model
# ------------------------------------------------------------
model_name = "mistralai/Mistral-7B-Instruct-v0.2"
tokenizer = AutoTokenizer.from_pretrained(model_name)

# bitsandbytes 4 bit quantisation dramatically reduces VRAM
```

```
quantization_config = {"load_in_4bit": True,
                       "bnb_4bit_compute_dtype": torch.float16,
                       "bnb_4bit_use_double_quant": True}
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    device_map="auto",
    **quantization_config
)

model.eval()   # inference mode

# ----------------------------------------------------------
# 4þ  ã (Optional) Load a LoRA adapter if you have one
# ----------------------------------------------------------
# adapter_path = "./lora-mistral-medium"
# if os.path.isdir(adapter_path):
#     model = PeftModel.from_pretrained(model, adapter_path)

# ----------------------------------------------------------
# 5þ  ã Define a helper for chat  style prompting
# ----------------------------------------------------------
def chat_completion(system_prompt: str, user_prompt: str,
                    temperature: float = 0.7,
                    max_new_tokens: int = 256,
                    top_p: float = 0.9,
                    top_k: int = 40):
    # Build a single string according to the model's expected format
    # Mistral  Instruct uses <s>[INST] … [/INST] format
    full_prompt = f"""<s>[INST] <<SYS>>\n{system_prompt}\n<</SYS>>\n\n{user_prompt} [/INST]"""
    inputs = tokenizer(full_prompt, return_tensors="pt").to(model.device)

    with torch.no_grad():
        output_ids = model.generate(
            **inputs,
            do_sample=True,
            temperature=temperature,
            top_p=top_p,
            top_k=top_k,
            max_new_tokens=max_new_tokens,
            pad_token_id=tokenizer.eos_token_id,
            eos_token_id=tokenizer.eos_token_id,
        )
    # Strip the prompt part to keep only the model's answer
    answer = tokenizer.decode(output_ids[0][inputs["input_ids"].shape[-1]:],
                              skip_special_tokens=True)
    return answer.strip()

# ----------------------------------------------------------
# 6þ  ã Try it out!
# ----------------------------------------------------------
system = "You are a friendly AI assistant specialized in explaining machine  learning concepts."
user = "Explain the difference between supervised fine  tuning and RLHF in simple terms."

print(chat_completion(system, user))
```

Explanation of key parts
- `load_in_4bit=True` loads the model in 4 bit quantised format ("H /40 /% VRAM of fp16).
- `device_map="auto"` shards the model across available GPUs/CPU.

- Prompt format `<s>[INST] … [/INST]` is required by Mistral Instruct.
- `do_sample=True` + `temperature/top_p/top_k` give stochastic, creative outputs.
- `PeftModel` enables LoRA adapters without touching the base weights.

> **Tip:** If you only have a single RTX /3060 (12 /GB), the 4 bit version of a 7 /B model should fit, albeit with somewhat slower generation.

7þ ã Ethical, Legal & Technical Challenges
Challenge	Why it matters	Mitigation strategies
Bias & Toxicity	Training data contains stereotypes, hate speech.	Use moderation filters. Fine tune on curated “safe” datasets.
Hallucination	Model may fabricate authoritative sounding facts.	Retrieval augmented generation (RAG). Post hoc fact checking tools.
Compute & Carbon Footprint	Training a trillion parameter model "H /hundreds of MWh.	Use efficient architectures (MoE, Mixture of Experts). Reuse open checkpoints.
Data Privacy	Public crawls may contain personal data.	Legal compliant scrubbing (GDPR). Employ differential privacy training if required.
Misuse	Deepfakes, spam, disinformation.	Controlled API access, rate limiting. Watermarking model outputs.
Intellectual Property	Generating copyrighted text/code.	Provide attribution mechanisms. Offer opt out options for copyrighted content.

8þ ã Future Directions (2025 2026 Outlook)
Trend	Expected impact
Retrieval Augmented Generation (RAG) + Vector Stores	Near real time knowledge, cheaper than massive constant size pre training.
Multimodal “All in One” models	Unified vision language audio models (e.g., Gemini /1.5 Pro) that can process video, code, and speech in the same forward pass.
Sparse Mixture of Experts (MoE) at scale	Trillion parameter effective capacity while keeping inference cost low (e.g., Mixtral 8×7B, Google’s GLaM 2).
Federated & Privacy Preserving LLMs	Training on user devices without central data collection, leveraging Secure Aggregation.
Self Supervised RL (e.g., “Self Play” for reasoning)	Models that improve their own problem solving ability without human labels.
Standardised “AI Fact Sheets	Industry wide reporting of model size, data provenance, benchmarks, and carbon impact.

Ø=ÜÚ Quick Reference Cheat Sheet
Concept	Symbol / Formula	Typical Values
Parameter count	*P*	7 /M – 1 /T
Context length	*L*	4 /k – 32 /k tokens
Training compute	*C* (GPU hours)	10 t – 10 v
Top p (nucleus) sampling	keep tokens where :7 Ö" &B § ¢ Â ã’ b ã“R À
Temperature (<B’ Â 6ögFÖ ,†Æöv—G2óÄ)	0.2 (deterministic) !’ 1.2 (creative)
LoRA rank	*r*	4 – 64 (depends on task)
Quantisation bits	*b*	4 bit (GPTQ/AWQ) or 8 bit

Next Steps for You

Goal	Action
Understand theory	Read *“Attention is All You Need”* (Vaswani 2017) + *“Scaling Laws”* (Kaplan 2020).
Run your first inference	Follow the code snippet above on a local GPU or Colab.
Experiment with prompting	Try zero shot, few shot, chain of thought on the same model.
Fine tune cheaply	Use LoRA + `peft`, train on a small domain dataset (e.g., medical Q&A).
Deploy	Spin up a `vllm` server or use HuggingFace Inference Endpoints for API access.
Stay safe	Add content moderation pipelines (e.g., OpenAI moderation API or `toxicity` classifiers).

Happy LLM learning! Ø=Þ€
If you’d like deeper tutorials—like a full LoRA fine tuning walkthrough, building a RAG system, or exploring multimodal models—just let me know!